

Chapter 06 — API, Controller & Data Layer

Component type: Technical — the HTTP surface + persistence. **Source:**

```
backend/src/controllers/analysis.controller.ts , routes/repoAnalysis.route.ts ,  
dao/analysis.dao.ts , models/repoAnalysis.model.ts , app.ts , server.ts , config/config.ts
```

1. The endpoint

POST /api/analysis (**body:** { `repoUrl` })

Full controller flow (`analyzeRepo`):

```
1. validate repoUrl is a non-empty string → else 400  
2. cleanUrl = trim; repoUrlHash = SHA256(cleanUrl)  
3. cache check: analysisDAO.findByUrlHash(repoUrlHash)  
   hit → 200 { cached:true, data: cached.result, analysisId } ← return early  
4. create record { userId:null, repoUrl, repoUrlHash, jobId: randomUUID } (status "waiting")  
   create fails → 500 "Failed to initialise analysis record"  
5. markActive(id)  
6. aiResult = aiService.analyseRepository(cleanUrl) ← the whole pipeline (Ch 04/05)  
7. !success → markFailed(id, errors.join(" | ")) → 500 { errors }  
8. success → markCompleted(id, result) → 200 { cached:false, data, analysisId, warnings? }
```

Caching is keyed by `SHA256(repoUrl)` — identical URLs return the stored result instantly (no GitHub/LLM cost). Note this means a re-pushed repo returns a **stale** analysis until the cache entry is bypassed (see `plan.md` P2.2 `force` flag).

2. Response shapes

Success

```
{ "success": true, "cached": false, "message": "Analysis completed",  
  "data": { "m1": [...], "m2": {...}, "m3": {...} },  
  "analysisId": "<mongo id>", "warnings": [...optional...] }
```

`warnings` is present only when the pipeline produced non-fatal errors (e.g. one module degraded).

Cached — same but `cached:true`, no `warnings`. **Error** — { `"success": false, "message": "Analysis failed", "errors": [...]` } (HTTP 500), or { `success:false, message:"repoUrl is required"` } (HTTP 400).

3. Data layer

3.1 DAO (`analysis.dao.ts`)

Thin persistence API used by the controller:

- `findByUrlHash(hash)` — cache lookup (completed record for that URL).
- `create({...})` — insert a new `waiting` record.
- `markActive(id)` / `markCompleted(id, result)` / `markFailed(id, error)` — status transitions.

3.2 Mongoose model (`repoAnalysis.model.ts`)

Document `IRepoAnalysis`:

Field	Type	Notes
<code>userId</code>	<code>ObjectId?</code>	nullable, indexed (auth not wired yet)
<code>repoUrl</code>	<code>string</code>	trimmed
<code>repoUrlHash</code>	<code>string</code>	indexed (cache key)
<code>jobId</code>	<code>string</code>	unique
<code>status</code>	<code>enum</code>	<code>waiting</code> <code>active</code> <code>completed</code> <code>failed</code> , indexed
<code>result</code>	<code>IAalysisResult null</code>	the M1/M2/M3 payload
<code>error</code>	<code>string?</code>	failure reason
<code>createdAt / completedAt</code>	<code>Date</code>	<code>timestamps:true</code>

Indexes: `{userId:1, createdAt:-1}`, `{repoUrlHash:1, status:1}`, `unique jobId`.

Schema bug to note: `result.m3` is declared as `[DependencyNodeSchema]` (an **array**), but the runtime `m3` is an **object** `{ graph, formattedAscii }`. So the graph array may persist under the wrong shape and `formattedAscii` is dropped. Tracked in `plan.md`.

4. App & server wiring

- `server.ts` — entry point: loads config, connects Mongo (`db.ts`), starts Express on `PORT` (default env, docs use **4000**).
 - `app.ts` — middleware: `cors()` (open — restrict in prod), `morgan` (HTTP logs), `express.json()`; mounts the analysis router.
 - `config/config.ts` — **throws on startup** if any of `PORT`, `MONGO_URI`, `GITHUB_TOKEN`, `NVIDIA_API_KEY` is missing. Loads `.env` from backend dir then workspace-root fallbacks.
-

5. Known gaps / improvement ideas

- **No `repoUrl` format validation** beyond "is a string" — a Zod/GitHub-domain check belongs here.
- **Open CORS + no rate limiting** → abuse risk on the GitHub/LLM-cost path.
- **No auth** — `userId` is always `null`.
- **Synchronous request** — the client waits for the full pipeline; no job/polling API despite the `jobId` / `status` machinery existing.
- Fix the `m3` array-vs-object schema mismatch so `formattedAscii` persists.